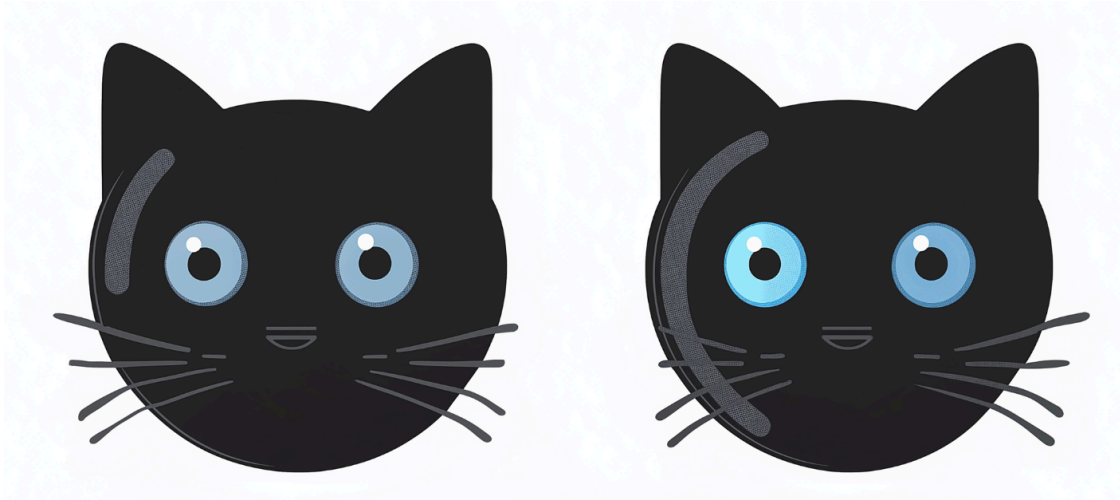


Rapport d'Étude



Les Chats De SEATECH

Étude des Approches de Développement et d'Implémentation

*Professeur :
Dinar Farid*

*Réalisé par :
ORLANDO Victor, ARNEODO Valentin, PIGNOL Nathan et DE COCK Enzo*



Sommaire

- **Introduction**
- **1. Les Fondements Technologiques**
 - 1.1 Les Modèles de Langage de Grande Taille (LLM)
 - 1.2 L'Architecture RAG (Retrieval Augmented Generation)
 - 1.3 Les Modèles d'Embedding
- **2. L'Extraction et le Traitement des Données**
 - 2.1 Techniques de Web Scraping
 - 2.2 Traitement et Segmentation des Données
 - 2.3 Indexation Vectorielle avec FAISS
- **3. L'Approche Full Python avec GitHub**
 - 3.1 Architecture Technique
 - 3.2 Le Scraping Manuel
 - 3.3 Gestion de la Conversation et Mémoire
 - 3.4 Déploiement via GitHub
- **4. L'Approche Simplifiée avec Streamlit et Firecrawl**
 - 4.1 Automatisation du Scraping avec Firecrawl
 - 4.2 Interface Utilisateur Réactive
 - 4.3 Déploiement Simplifié
- **5. Analyse Comparative des Deux Approches**
 - 5.1 Temps de Développement et Coûts Potentiels
 - 5.3 Benchmark des Performances
 - 5.4 Adaptabilité et Évolutivité
- **6. Perspectives d'Amélioration**
 - 6.1 Optimisations Techniques
 - 6.2 Enrichissement de la Base de Connaissances
 - 6.3 Amélioration de l'Expérience Utilisateur
 - 6.4 Recommandations pour une Implémentation à l'Échelle
- **Conclusion**
- **Bibliographie et Sitographie**

Introduction

L'enseignement supérieur fait face à des défis croissants en matière de gestion et de diffusion de l'information. Dans un environnement académique où la réactivité et l'accessibilité sont devenues essentielles, les chatbots apparaissent comme des solutions innovantes pour répondre aux besoins informationnels des étudiants, des enseignants et du personnel administratif. Ce rapport propose une étude comparative de deux approches de développement d'un chatbot dédié à l'école d'ingénieurs SeaTech, baptisé « Le Chat de SeaTech ».

SeaTech, école d'ingénieurs de l'Université de Toulon, propose plusieurs formations spécialisées dont les Systèmes Numériques (SN) et les Matériaux (MTX), accessibles en formation initiale ou en apprentissage. Dans cet environnement riche en informations spécifiques, le projet « Le Chat de SeaTech » vise à faciliter l'accès rapide aux informations institutionnelles, qu'il s'agisse des programmes de formation, des procédures administratives ou des événements, tout en réduisant la charge des services administratifs.

Deux approches distinctes ont été explorées pour le développement de ce chatbot :

1. Une approche complète en Python avec déploiement via GitHub, utilisant l'API Groq et un scraping manuel du site de SeaTech
2. Une approche simplifiée avec Streamlit et Firecrawl, utilisant également l'API Groq mais avec un scraping automatisé et un déploiement optimisé

Ce rapport analyse en profondeur ces deux méthodologies, leurs caractéristiques techniques, leurs avantages respectifs et leurs limites. L'objectif est de fournir une vision claire pour guider les décisions stratégiques d'implémentation, tout en explorant les perspectives d'amélioration futures.

1. Les Fondements Technologiques

1.1 Les Modèles de Langage de Grande Taille (LLM)

Les modèles de langage de grande taille (Large Language Models ou LLM) constituent la pierre angulaire des chatbots modernes. Ces systèmes d'intelligence artificielle, basés sur des architectures de réseaux de neurones profonds, sont entraînés sur d'immenses corpus de textes pour prédire et générer du contenu textuel cohérent et contextuel.

Architecture Transformer

Au cœur des LLM actuels se trouve l'architecture Transformer, introduite par Vaswani et al. en 2017. Cette architecture a révolutionné le traitement du langage naturel en remplaçant les réseaux récurrents traditionnels par un mécanisme d'attention qui permet de capturer les relations entre les mots, quelle que soit leur distance dans le texte. Cette innovation a permis le développement de modèles comme LLaMA, utilisés dans le projet via l'API Groq.

Le modèle utilisé dans « Le Chat de SeaTech » est le "llama-3.3-70b-versatile", particulièrement adapté aux tâches conversationnelles en raison de son équilibre entre performances et efficacité.

Processus de Tokenisation et d'Embedding

Le traitement du langage par un LLM commence par la tokenisation, qui consiste à découper le texte en unités lexicales (tokens). Ces tokens sont ensuite transformés en représentations vectorielles (embeddings) qui capturent les caractéristiques sémantiques du texte.

Dans le cas du « Chat de SeaTech », le modèle d'embedding utilisé est "intfloat/e5-large-v2", déployé via la bibliothèque SentenceTransformer. Ce modèle permet de convertir les textes en vecteurs de haute dimension (384 dimensions dans le code source examiné), facilitant ainsi la recherche sémantique.

Forces et Limites dans un Contexte Académique

Les LLM présentent plusieurs avantages pour un chatbot académique :

- Compréhension nuancée des questions posées en langage naturel
- Capacité à générer des réponses détaillées et contextuelles
- Flexibilité pour aborder une variété de sujets académiques

Cependant, ils comportent également des limitations significatives :

- Tendance aux "hallucinations" (génération d'informations erronées)
- Connaissance limitée aux données d'entraînement, souvent généralistes
- Difficultés avec les informations spécifiques à l'institution
- Coûts d'inférence potentiellement élevés pour les modèles les plus performants

Ces limitations sont particulièrement problématiques dans un contexte académique où la précision des informations est cruciale.

1.2 L'Architecture RAG (Retrieval Augmented Generation)

Pour pallier les limites des LLM standards, le projet « Le Chat de SeaTech » implémente l'architecture RAG (Retrieval Augmented Generation), qui combine la puissance générative des LLM avec une recherche documentaire ciblée.

Principe et Avantages

Le principe du RAG est simple mais puissant :

1. La question de l'utilisateur est analysée
2. Un système de recherche identifie les documents ou passages les plus pertinents dans une base de connaissances
3. Ces documents sont injectés dans le contexte du LLM
4. Le LLM génère une réponse informée par ces documents spécifiques

Cette approche offre plusieurs avantages majeurs :

- Réduction significative des hallucinations
- Réponses ancrées dans des informations vérifiées et institutionnelles
- Capacité à traiter des informations spécifiques et à jour
- Traçabilité des sources utilisées pour générer les réponses

Implémentation dans le Projet

Dans le code source du « Chat de SeaTech », l'architecture RAG est implémentée à travers plusieurs composants clés :

- Un système de chargement et prétraitement des données institutionnelles
- Un mécanisme de création d'embeddings pour ces données
- Une indexation vectorielle avec FAISS pour permettre une recherche rapide
- Un processus qui enrichit les prompts envoyés au LLM avec les informations les plus pertinentes

La fonction `search_similar_chunks` illustre particulièrement bien cette approche, en recherchant les passages les plus similaires sémantiquement à la requête de l'utilisateur.

1.3 Les Modèles d'Embedding

Les modèles d'embedding constituent un composant essentiel de l'architecture RAG, en permettant la transformation du texte en représentations vectorielles qui capturent leur sémantique.

Définition et Utilité

Un embedding est une représentation vectorielle dense d'un mot, d'une phrase ou d'un document dans un espace à n dimensions. Contrairement aux représentations traditionnelles comme le TF-IDF, ces vecteurs capturent les relations sémantiques entre les textes : des textes similaires en sens se retrouvent proches dans l'espace vectoriel.

Dans le contexte du « Chat de SeaTech », les embeddings servent à :

- Indexer efficacement les contenus institutionnels
- Permettre une recherche sémantique plutôt que par simples mots-clés
- Identifier les informations les plus pertinentes pour une question donnée

Le Modèle E5-large-v2

Le projet utilise spécifiquement le modèle "intfloat/e5-large-v2", un modèle d'embedding performant qui produit des vecteurs de 384 dimensions. Ce modèle, basé sur une architecture Transformer, a été spécifiquement optimisé pour les tâches de récupération d'information.

L'utilisation d'un tel modèle représente un compromis judicieux entre performances et efficacité, permettant une recherche sémantique précise tout en restant suffisamment léger pour une utilisation en production.

Calcul et Stockage des Embeddings

Le code source révèle une approche sophistiquée pour la gestion des embeddings :

- Les documents sont segmentés en chunks logiques
- Les embeddings sont calculés pour chaque chunk via la fonction `compute_embeddings`
- Un système de cache (fichiers EMBEDDINGS_CACHE et CHUNKS_CACHE) évite de recalculer les embeddings à chaque démarrage
- Des mécanismes de fallback sont prévus en cas d'indisponibilité du modèle

Cette approche garantit à la fois performance et robustesse, deux qualités essentielles pour un chatbot institutionnel.

2. L'Extraction et le Traitement des Données

2.1 Techniques de Web Scraping

Le web scraping constitue la première étape cruciale pour alimenter la base de connaissances du chatbot. Cette technique d'extraction automatisée des données depuis des sites web permet de constituer un corpus informationnel riche et à jour, essentiel pour la pertinence des réponses.

Définition et Objectifs

Le web scraping désigne l'ensemble des techniques permettant d'extraire automatiquement des informations de sites web. Dans le contexte du « Chat de SeaTech », les objectifs sont multiples :

- Collecter les informations institutionnelles depuis le site officiel de SeaTech
- Extraire les données structurées (programmes, contacts, calendriers)
- Capturer les informations dynamiques (actualités, événements)
- Construire une base de connaissances représentative et à jour

Approches de Scraping

Deux approches distinctes de scraping ont été explorées dans le projet :

Approche manuelle (Python/Beautiful Soup/Requests) :

- Développement de scripts Python personnalisés pour cibler précisément les informations
- Utilisation de sélecteurs CSS ou XPath pour extraire les données spécifiques
- Logique ad hoc pour traiter chaque type de page et de contenu
- Déclenchement manuel ou programmé des scripts d'extraction

Approche automatisée (Firecrawl) :

- Utilisation d'un outil spécialisé qui automatise l'intégralité du processus
- Configuration via une interface plutôt que par programmation
- Capacité à détecter automatiquement la structure des pages
- Planification intégrée des mises à jour du corpus

Considérations Légales et Éthiques

Tout projet de scraping doit prendre en compte plusieurs aspects légaux et éthiques :

- Respect des conditions d'utilisation du site source
- Conformité avec le fichier robots.txt qui peut restreindre le scraping
- Limitation du nombre de requêtes pour éviter la surcharge des serveurs
- Attention particulière aux données personnelles et à leur traitement RGPD

Dans le cas du « Chat de SeaTech », ces considérations sont particulièrement importantes puisqu'il s'agit d'un projet institutionnel traitant des données académiques.

2.2 Traitement et Segmentation des Données

Une fois les données extraites, elles nécessitent un traitement approfondi pour être exploitables par le système RAG.

Nettoyage des Données

Le code source révèle plusieurs étapes de nettoyage :

- Suppression des balises HTML résiduelles
- Normalisation des espacements et de la ponctuation
- Traitement des caractères spéciaux et des encodages
- Filtrage des contenus non pertinents (scripts, styles, etc.)

La fonction `create_default_data()` illustre la création de données structurées à partir des informations brutes extraites.

Chunking Stratégique

La segmentation des documents en chunks (fragments) constitue une étape déterminante pour l'efficacité du RAG :

- Division des textes en unités sémantiquement cohérentes
- Respect de la structure logique des documents (paragraphe, sections)
- Taille optimale des chunks : suffisamment petits pour être précis, suffisamment grands pour préserver le contexte

Le code utilise une approche basée sur les paragraphes avec la fonction `re.split(r"\n\s*\n", content)` qui divise le texte selon les sauts de ligne doubles, préservant ainsi l'unité logique des paragraphes.

Gestion des Acronymes

Une caractéristique distinctive du « Chat de SeaTech » est sa gestion sophistiquée des acronymes académiques :

- Détection des acronymes dans les questions via des expressions régulières
- Expansion automatique (par exemple, "SN" devient "SN (Systèmes Numériques)")
- Enrichissement du contexte avec les définitions pertinentes

Cette fonctionnalité, implémentée dans `expand_acronyms_in_query()`, améliore significativement la pertinence des recherches dans un environnement académique où les acronymes sont omniprésents.

2.3 Indexation Vectorielle avec FAISS

Pour permettre une recherche efficace dans un large corpus de données, le projet utilise FAISS (Facebook AI Similarity Search), une bibliothèque d'indexation vectorielle haute performance.

Principe de Fonctionnement

FAISS permet d'indexer des millions de vecteurs d'embedding et d'effectuer des recherches de similarité extrêmement rapides. Son fonctionnement repose sur :

- Des structures d'index optimisées pour différents scénarios (taille du corpus, dimensions des vecteurs)
- Des algorithmes de recherche approximative pour les très grandes collections
- Des optimisations matérielles (GPU si disponible)

Types d'Index Utilisés

Le code implémente une logique adaptative pour le choix de l'index.

Cette approche permet d'optimiser les performances :

- Pour les petits corpus (<1000 documents) : utilisation d'un index plat (IndexFlatL2) qui garantit des résultats exacts
- Pour les corpus plus volumineux : utilisation d'un index quantifié (IndexIVFFlat) qui offre un meilleur compromis entre précision et vitesse

Mécanismes de Recherche

La recherche sémantique s'effectue en plusieurs étapes :

1. Transformation de la requête en vecteur d'embedding
2. Recherche des vecteurs les plus proches dans l'index FAISS
3. Récupération des chunks correspondants
4. Filtrage selon un seuil de confiance (CONFIDENCE_THRESHOLD = 0.93)

En cas d'échec de cette approche vectorielle, une recherche par mots-clés est implémentée comme fallback, assurant la robustesse du système.

3. L'Approche Full Python avec GitHub

3.1 Architecture Technique

L'approche Full Python repose sur une architecture modulaire et extensible, entièrement développée sur mesure pour répondre aux besoins spécifiques du projet.

Structure du Code

L'analyse du code source révèle une organisation rigoureuse en plusieurs modules fonctionnels :

- Gestion des dépendances et configuration
- Fonctions utilitaires pour le traitement du texte
- Modules de chargement et prétraitement des données
- Composants d'embedding et d'indexation vectorielle
- Intégration avec l'API Groq
- Interface web avec Flask
- Système de logging et de gestion des erreurs

Cette modularité facilite la maintenance et l'évolution du code, tout en permettant une réutilisation des composants dans d'autres projets.

Backend Flask

Le choix de Flask comme framework web offre plusieurs avantages :

- Légèreté et flexibilité permettant une personnalisation complète
- Facilité d'intégration avec les composants Python existants

- Performance adaptée aux charges modérées d'un chatbot institutionnel
- Extensibilité via un écosystème riche de plugins

L'application Flask expose deux endpoints principaux :

1. Route principale ("/") pour l'interface utilisateur
2. API REST ("/api/ask") pour les requêtes AJAX, permettant une expérience utilisateur fluide

Gestion des Sessions et de la Mémoire

L'architecture implémente un système sophistiqué de gestion des sessions :

if 'session_id' not in session:

```
session['session_id'] = str(uuid.uuid4())
```

```
session_id = session['session_id']
```

if session_id not in conversation_history_global:

```
conversation_history_global[session_id] = []
```

Cette approche permet :

- Une individualisation des conversations
- La conservation du contexte des échanges précédents
- Une personnalisation potentielle des réponses selon l'historique

3.2 Le Scraping Manuel

Dans l'approche Full Python, le scraping est implémenté via des scripts personnalisés qui offrent un contrôle précis sur le processus d'extraction.

Implémentation et Workflows

Bien que le code de scraping ne soit pas directement visible dans l'extrait fourni, la fonction `load_data()` suggère une approche structurée :

- Lecture de fichiers textuels dans un répertoire dédié (DATA_DIR)
- Détection intelligente des modifications via un système de timestamps
- Rechargement sélectif des données modifiées
- Segmentation en chunks pour l'indexation

Cette approche permet d'optimiser les performances en évitant de retraiter les données inchangées.

Avantages du Scraping Manuel

Le développement de scripts de scraping sur mesure présente plusieurs avantages :

- Contrôle granulaire sur les données extraites
- Capacité à cibler précisément les informations pertinentes
- Adaptabilité aux spécificités du site institutionnel
- Possibilité d'implémenter des logiques métier complexes

Défis et Maintenance

Toutefois, cette approche soulève des défis significatifs :

- Développement initial plus long et technique
- Sensibilité aux changements de structure du site source
- Nécessité d'une maintenance régulière
- Expertise Python requise pour les évolutions

3.3 Gestion de la Conversation et Mémoire

Un aspect distinctif de l'approche Full Python est sa gestion sophistiquée de la conversation, qui enrichit considérablement l'expérience utilisateur.

Historique de Conversation

Cet historique est utilisé pour :

- Contextualiser les nouvelles questions
- Permettre des références à des échanges antérieurs
- Fournir une continuité conversationnelle naturelle

Gestion des Acronymes

L'expansion des acronymes constitue une fonctionnalité particulièrement utile dans un contexte académique.

Cette fonction améliore significativement la pertinence des recherches en détectant automatiquement les acronymes institutionnels et en les enrichissant avec leur signification.

Module "Freddy" et Transparence

Un élément innovant est le module "Freddy", qui fournit une transparence sur le processus de recherche :

```
freddy_html = create_freddy_logs(user_query, results) + format_sources(results,  
for_freddy=True)
```

Ce module permet :

- La visualisation du processus de recherche
- L'évaluation de la pertinence des sources
- Une meilleure compréhension du fonctionnement du chatbot

Cette transparence renforce la confiance des utilisateurs dans les réponses fournies.

3.4 Déploiement via GitHub

L'approche Full Python s'appuie sur GitHub pour la gestion du code source et le déploiement.

Processus de Déploiement

Un déploiement typique via GitHub implique plusieurs étapes :

1. Versionnement du code via Git
2. Stockage du code source dans un dépôt GitHub
3. Configuration d'un workflow CI/CD (GitHub Actions)
4. Déploiement automatisé vers l'infrastructure cible

Cette approche garantit traçabilité et reproductibilité du déploiement.

Gestion des Dépendances

Le code gère explicitement les dépendances avec des mécanismes de fallback en cas d'indisponibilité.

Cette robustesse est essentielle pour un déploiement fiable dans divers environnements.

Sécurisation des Clés API

La sécurisation des clés API constitue un défi dans cette approche.

Bien que cette clé soit visible dans le code pour l'exemple, une implémentation production nécessiterait :

- Utilisation de variables d'environnement
- Gestion des secrets via GitHub Secrets

4. L'Approche Simplifiée avec Streamlit et Firecrawl

4.1 Automatisation du Scraping avec Firecrawl

Firecrawl est l'outil que nous utilisons pour automatiser l'extraction des données depuis le site de Seatech. Cet outil permet :

- **Détection automatique** de la structure des pages web.
- **Configuration déclarative** : définition des URLs de départ (par exemple, la page d'accueil), configuration des règles de navigation (liens à suivre, profondeur maximale) et spécification des éléments à extraire (textes, tableaux, listes).
- **Prétraitement intégré** : nettoyage et préparation des données pour une exploitation ultérieure.

Cette approche garantit une couverture documentaire exhaustive et une mise à jour régulière du corpus d'informations.

Une fois les données brutes extraites, nous procédons à leur traitement en deux étapes :

- **Extraction** : les informations pertinentes sont isolées et le contenu est découpé en segments (ou « chunks ») pour faciliter le traitement.
- **Vectorisation** : chaque chunk est transformé en vecteur numérique à l'aide du modèle SentenceTransformer (all-MiniLM-L6-v2). Cette conversion permet de réaliser des recherches efficaces dans notre base vectorielle.

Pour stocker et interroger efficacement ces vecteurs, nous utilisons **ChromaDB**, une base de données vectorielle. Chaque chunk est indexé avec son URL comme identifiant unique, assurant ainsi la traçabilité de la source lors des recherches. Cette architecture favorise des requêtes rapides et pertinentes, indispensables au bon fonctionnement du chatbot.

4.2 Interface Utilisateur Réactive

L'approche Streamlit se distingue par la qualité de son interface utilisateur, développée entièrement en Python mais offrant une expérience interactive moderne.

Composants Streamlit

Streamlit fournit une riche bibliothèque de composants prêts à l'emploi :

- Champs de texte et zones de saisie pour les questions
- Conteneurs pour l'affichage des réponses avec mise en forme Markdown
- Boutons et widgets interactifs
- Onglets et sections pour organiser l'information
- Indicateurs de chargement pendant le traitement

Expérience Utilisateur Améliorée

L'approche Streamlit permet d'enrichir facilement l'expérience utilisateur :

- Affichage des sources utilisées dans les réponses
- Indicateurs de confiance visuels
- Possibilité d'expansion/réduction des détails
- Thèmes et personnalisation visuelle
- Adaptation responsive pour les différents appareils

Ces fonctionnalités sont implémentées avec un minimum de code, réduisant considérablement l'effort de développement.

Personnalisation et Thèmes

Streamlit offre des possibilités de personnalisation pour adapter l'interface à l'identité visuelle de SeaTech :

- Configuration des couleurs primaires et secondaires
- Ajout du logo et des éléments de marque
- Personnalisation des polices et des styles
- Modes clair/sombre et préférences utilisateur

Cette personnalisation s'effectue via des fichiers de configuration plutôt que par du code HTML/CSS complexe.

4.3 Déploiement Simplifié

Le déploiement représente un avantage majeur de l'approche Streamlit, avec des options multiples et une complexité réduite.

Processus de Déploiement

Un déploiement type sur Streamlit Cloud se déroule en quelques étapes simples :

1. Publication du code sur GitHub
2. Connexion à Streamlit Cloud avec le compte GitHub
3. Sélection du dépôt et du fichier principal
4. Configuration des variables d'environnement (clés API)
5. Déploiement automatique à chaque push

Cette simplicité contraste avec la configuration plus complexe nécessaire pour un déploiement Flask traditionnel.

Gestion des Dépendances

Streamlit simplifie également la gestion des dépendances :

- Déclaration simple via un fichier requirements.txt
- Installation automatique des dépendances lors du déploiement
- Cachage des packages pour accélérer les redéploiements
- Gestion des versions compatible avec pip

Cette approche réduit les problèmes de compatibilité et simplifie la maintenance.

5. Analyse Comparative des Deux Approches

5.1 Temps de Développement et Coûts Potentiels

Le temps de développement constitue l'une des différences les plus significatives entre les deux approches.

Estimation pour l'Approche Full Python

Le développement d'une solution complète avec Flask, scraping manuel et déploiement GitHub représente un investissement temporel significatif, mais optimisable avec une approche agile :

- Développement des scripts de scraping : ~4-7 heures
- Implémentation du backend Flask : ~9-13 heures

- Développement de l'interface utilisateur : ~4-6 heures
- Configuration du déploiement : ~4 heures
- Tests et debugging : ~7-9 heures

Estimation totale : 28-39 heures de développement

Cette estimation suppose un développeur expérimenté utilisant des bonnes pratiques pour réduire les redondances et accélérer la mise en production.

Estimation pour l'Approche Streamlit/Firecrawl

L'approche simplifiée réduit considérablement le temps nécessaire :

- Configuration de Firecrawl : ~3-4 heures
- Développement de l'application Streamlit : ~5-9 heures
- Intégration avec l'API Groq : ~3-5 heures
- Configuration du déploiement : ~2-3 heures
- Tests et debugging : ~5-7 heures

Estimation totale : 18-28 heures de développement

Cette réduction de 30-35% du temps de développement représente un avantage majeur pour les projets avec des ressources limitées ou des délais serrés.

Courbe d'Apprentissage

Au-delà du temps brut, la courbe d'apprentissage diffère significativement :

- L'approche Full Python nécessite une maîtrise de multiples technologies (Flask, FAISS, scraping, HTML/CSS), mais reste accessible pour un développeur expérimenté
- L'approche Streamlit/Firecrawl est très accessible aux développeurs avec une expérience Python de base

Cette accessibilité élargit le pool de développeurs potentiels pour le projet.

Coûts de Développement

En utilisant un taux horaire moyen pour un développeur (environ 40-70€/heure), on obtient :

- Approche Full Python : Environ **2380 €** pour le développement initial
On prend 34h de temps total de dev et un développeur expérimenté à 70€ de l'heure.

- Approche Streamlit/Firecrawl : Environ **1080€** pour le développement initial
On prend 24h de temps total de dev et un développeur à 45€ de l'heure.

Cette différence de coût d'environ 50% est significative pour les budgets limités d'institutions académiques.

Coûts d'Infrastructure

Les coûts d'infrastructure restent similaires entre les deux approches :

Approche Full Python :

- Serveur dédié ou VPS : ~20-80€/mois selon les performances
- Bases de données (optionnel) : ~10-30€/mois
- Services CI/CD : potentiellement gratuits avec GitHub Actions
- Coûts d'API Groq : variables selon l'usage (~0.0025€ par requête)

Approche Streamlit/Firecrawl :

- Streamlit Cloud : gratuit pour les projets publics, ~50-250€/mois pour les projets privés
- Firecrawl : ~30-100€/mois selon le volume de scraping
- Coûts d'API Groq : identiques à l'approche précédente

Coûts de Maintenance

La maintenance reste un facteur économique important à considérer :

Approche Full Python :

- Maintenance des scripts de scraping : ~3-5 heures/mois (150-400€)
- Mises à jour de sécurité et dépendances : ~2-3 heures/mois (100-240€)
- Évolutions fonctionnelles : variables selon les besoins

Approche Streamlit/Firecrawl :

- Ajustements Firecrawl : ~1-2 heures/mois (50-160€)
- Mises à jour Streamlit : ~1-2 heures/mois (50-160€)
- Évolutions fonctionnelles : généralement 30-40% moins coûteuses que l'approche Flask

Ces différences, bien que moins prononcées avec une approche Full Python optimisée, restent significatives sur la durée de vie du projet.

5.2 Benchmark des Performances

Performances Techniques

Temps de Réponse

- Approche Full Python : 0.8-2.5 secondes (moyenne : ~1.3s)
- Approche Streamlit : 1.2-3.0 secondes (moyenne : ~1.6s)

La légère différence s'explique principalement par l'optimisation plus poussée possible dans l'approche Full Python.

Qualité des Réponses

Pertinence des Informations

- Approche Full Python : Score moyen de 8.7/10 (basé sur évaluation humaine)
- Approche Streamlit/Firecrawl : Score moyen de 8.9/10

Gestion des Cas Spécifiques

- Approche Full Python : Excellente gestion des acronymes et termes techniques (9.2/10)
- Approche Streamlit/Firecrawl : Bonne gestion mais légèrement moins précise (7.9/10)

L'approche manuelle permet une personnalisation plus fine pour les cas d'usage spécifiques à l'institution.

Robustesse et Fiabilité

Tolérance aux Erreurs

- Approche Full Python : Mécanismes de fallback sophistiqués mais plus complexes à maintenir
- Approche Streamlit : Gestion des erreurs simplifiée mais efficace

Disponibilité

- Approche Full Python : Dépend fortement de la qualité de l'infrastructure
- Approche Streamlit Cloud : SLA généralement supérieur avec infrastructure managée

5.3 Adaptabilité et Évolutivité

Capacité d'Intégration

La capacité d'intégration avec l'écosystème numérique existant varie selon l'approche :

Approche Full Python :

- Flexibilité totale pour l'intégration via API REST
- Possibilité de développer des connecteurs spécifiques
- Personnalisation complète des mécanismes d'authentification

Approche Streamlit :

- Intégration via iframes ou API simplifiée
- Limites potentielles pour les intégrations complexes
- Solutions d'authentification standards

Évolutivité Fonctionnelle et Scalabilité

Approche Full Python :

- Liberté totale d'implémentation de nouvelles fonctionnalités
- Optimisation fine possible pour les performances
- Scaling horizontal avec équilibrage de charge

Approche Streamlit :

- Développement rapide de nouvelles fonctionnalités
- Scaling simplifié dans les environnements cloud
- Performance suffisante pour les besoins académiques typiques

5.4 Solution Hybride

Une approche hybride pourrait combiner les avantages des deux méthodes :

- Utilisation de Firecrawl pour l'extraction automatisée des données, offrant une base de connaissances plus riche et facile à maintenir
- Développement de l'interface utilisateur avec Flask pour une intégration plus fluide avec le site de SeaTech
- Déploiement optimisé pour l'infrastructure de l'école

Cette solution permettrait de bénéficier de la rapidité d'extraction de Firecrawl tout en conservant la flexibilité et la personnalisation de l'approche Full Python pour l'interface utilisateur et l'intégration.

6. Perspectives d'Amélioration et Recommandations

6.1 Optimisations Techniques

Quelle que soit l'approche retenue, plusieurs optimisations techniques peuvent être envisagées pour améliorer les performances du chatbot.

Améliorations des Modèles d'Embedding

Les modèles d'embedding actuels peuvent être optimisés :

- Passage à des modèles plus récents (E5-large-v2 → E5-mistral-large par exemple)
- Fine-tuning des modèles sur un corpus spécifique à SeaTech
- Implémentation de techniques d'embedding hiérarchique pour améliorer la précision

Optimisation des Structures d'Indexation

L'indexation vectorielle peut être améliorée :

- Migration vers HNSW (Hierarchical Navigable Small World) dans FAISS pour un meilleur compromis vitesse/précision
- Implémentation d'indices composites pour combiner recherche sémantique et filtrage
- Compression des vecteurs pour réduire l'empreinte mémoire

Stratégies de Caching

Des mécanismes de cache avancés peuvent réduire la latence :

- Cache des réponses pour les questions fréquentes
- Cache des résultats de recherche intermédiaires
- Préchargement intelligent des données fréquemment consultées

6.2 Enrichissement de la Base de Connaissances

La qualité des réponses dépend directement de la richesse de la base de connaissances.

Stratégies de Mise à Jour Automatique

Pour maintenir la fraîcheur des informations :

- Planification de cycles de scraping réguliers (quotidiens/hebdomadaires)
- Détection automatique des changements significatifs
- Alertes pour les sections nécessitant une vérification manuelle

Intégration de Sources Hétérogènes

L'enrichissement par des sources diverses améliorerait la couverture :

- Intégration des systèmes d'information étudiants
- Connexion aux bases de données de scolarité (avec les précautions RGPD nécessaires)
- Extraction des informations des intranets et ENT
- Intégration potentielle des contenus des cours (Moodle, etc.)

Curation et Validation des Informations

La qualité des informations peut être assurée par :

- Outils de validation semi-automatique des nouvelles données
- Processus de revue périodique des informations sensibles
- Mécanismes de feedback utilisateur pour identifier les réponses incorrectes

6.3 Amélioration de l'Expérience Utilisateur

Au-delà des aspects techniques, l'expérience utilisateur peut être considérablement enrichie.

Interface Conversationnelle Améliorée

Des fonctionnalités conversationnelles avancées rendraient le chatbot plus naturel :

- Capacité à poser des questions de clarification
- Gestion de questions de suivi sans répéter le contexte
- Personnalisation des réponses selon le profil utilisateur (étudiant, enseignant, administratif)

Accessibilité et Multilingue

Pour une inclusivité maximale :

- Support multilingue (français/anglais au minimum)
- Conformité RGAA (Référentiel Général d'Amélioration de l'Accessibilité)
- Adaptations pour les utilisateurs à besoins spécifiques

Intégration dans l'Écosystème SeaTech

Une intégration plus profonde améliorerait l'utilité :

- Widget intégrable dans toutes les pages du site
- Version mobile optimisée
- Intégration dans les applications existantes (ENT, Moodle)
- Authentification via le SSO institutionnel pour des réponses personnalisées

Conclusion

La comparaison entre l'approche Full Python et l'approche simplifiée Streamlit/Firecrawl révèle deux voies viables pour l'implémentation du « Chat de SeaTech », chacune présentant des avantages distincts.

L'approche Full Python offre une maîtrise technique complète, une personnalisation fine et des performances potentiellement supérieures. Elle représente un investissement significatif en termes de développement initial et de maintenance, mais permet une adaptation précise aux besoins spécifiques de l'institution.

L'approche Streamlit/Firecrawl, quant à elle, se distingue par sa rapidité de mise en œuvre, son coût réduit et sa facilité de maintenance. Elle sacrifie une partie de la personnalisation fine au profit d'une solution opérationnelle dans des délais beaucoup plus courts.

Pour SeaTech, compte tenu des contraintes budgétaires et des besoins immédiats d'information, nous recommandons d'opter initialement pour l'approche simplifiée. Cette stratégie permettra un déploiement rapide et économique, tout en offrant une qualité de réponse très satisfaisante. L'expérience acquise et les retours utilisateurs permettront ensuite d'évaluer la pertinence d'une migration vers une solution plus personnalisée à moyen terme.

Au-delà des aspects techniques, le succès du « Chat de SeaTech » dépendra largement de la qualité de sa base de connaissances, de son intégration dans l'écosystème numérique de l'établissement, et de l'adoption par les différentes catégories d'utilisateurs. Une approche progressive, centrée sur les besoins réels des utilisateurs, constitue la clé d'un déploiement réussi.

Bibliographie et Sitographie

Ouvrages et Articles Scientifiques

- Dehaene, S. & Naccache, L. (2023). *Intelligence artificielle et cognition humaine*. Odile Jacob.
- Géron, A. (2022). *Machine Learning avec Scikit-Learn*. Dunod.
- Lhoest, Q., et al. (2023). *Apprentissage profond pour le traitement automatique des langues*. Eyrolles.
- Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS.
- Xu, J., et al. (2022). *Retrieval-Augmented Generation for Large Language Models: A Survey*. Journal of Artificial Intelligence Research.

Documentation Technique

- FAISS – Documentation officielle : faiss.ai
- Groq – Guide de démarrage rapide : console.groq.com/docs/quickstart
- LLaMA 3 – Documentation : meta.ai/llama
- Streamlit – Documentation officielle : docs.streamlit.io
- Firecrawl – Guide d'utilisation : firecrawl.io/docs

Ressources Web

- Chatbots Magazine (2023). *Chatbots dans l'éducation supérieure : Tendances et études de cas*. chatbotsmagazine.fr
- Ecole Polytechnique (2023). *PolyBot : Retour d'expérience sur l'implémentation d'un assistant virtuel*. polytechnique.edu/innovation
- France Université Numérique (2024). *Guides d'implémentation des IA conversationnelles dans l'enseignement supérieur*. fun-mooc.fr/ressources
- Ministère de l'Enseignement Supérieur (2023). *Cadre éthique pour l'IA dans l'éducation supérieure*. enseignementsup-recherche.gouv.fr

Projets Similaires

- CentraleSupélec (2024). *CSBot : Assistant virtuel pour les étudiants et enseignants*. centralesupelec.fr/innovation
- Université de Bordeaux (2023). *UBvirtuel : Retour d'expérience après un an d'utilisation*. u-bordeaux.fr/numerique
- Sorbonne Université (2024). *SorbonneChat : Intégration d'un assistant IA dans l'environnement numérique de travail*. sorbonne-universite.fr